Physics-Informed Neural Networks, with No Data

Solve a physics equation with a neural network whose only teacher is the equation itself

PINN | No training data at all | Uses Weeks 7, 8 and 11

The task. How does heat spread along a metal rod when nothing has been measured? A physics-informed neural network, or PINN, can learn the answer with no data at all. Instead of comparing its guesses with measurements, it is trained to break the equation, the conditions at the ends and the starting state as little as possible. You will build one, show when it is right, and find where it goes wrong.

THE DATA

There is no dataset. In Week 9, each student gets their own version of an equation that describes change over space and time, such as the heat equation, with its own numbers, end conditions and starting state, worked out from their student number. To check your answers, you use an exact solution or write a classical step-by-step solver (finite differences).

WHAT YOU WILL DO

1. Write the classical solver and check it against a case where the exact answer is known.
2. Build a PINN: a network that takes a position and a time and gives, for example, the temperature. Train it to obey the equation at many sample points and to match the end and starting conditions. PyTorch works out the derivatives the equation needs.
3. Run experiments, changing one thing at a time: the number of sample points, the balance between the parts of the loss, the size of the network and the activation function.
4. Show where it fails: maps of the error over space and time, compared with the classical solver, and how long each method takes.

WHAT YOU CODE YOURSELF

Write **backpropagation** and an **optimiser step** for a small network and check them against PyTorch, including the derivative of the output with respect to the inputs, which the equation needs. Use **PCA via SVD** on the solution over time to find how many patterns describe it, and check it against NumPy.

IS IT FAIR AND SAFE?

There are no people in this data, so fairness becomes **reliability across the problem**: is the answer equally good everywhere? Measure the error near the ends, at early and late times, and around sudden changes. Improve the worst area with one change, such as more sample points there, and measure the cost elsewhere. Maps of where the equation is broken show where the network struggles. Write down when a PINN could be trusted instead of a proven solver.

QUESTIONS TO PREPARE FOR

Why does ReLU struggle here while tanh works? (Think about its second derivative.) With no data, how do you know your answer is right? When would you use the classical solver instead?

COMMON MISTAKES

A small loss mistaken for a correct answer. The end conditions ignored because the equation part of the loss is much larger. Comparing a PINN with a solver without counting how long each takes.

WEEK-BY-WEEK PLAN

Week 9: your version received; classical solver written and checked; first PINN trained. **Week 10:** experiments, one change at a time; hand-written pieces, including the input derivative, passing their tests. **Week 11:** error maps, reliability check, PCA via SVD, time comparison. **By 6 January 2027:** report written; repository checked and submitted. **Week of 11 January:** 10-minute in-person interview.

TO AIM HIGHER

Move sample points to where the network is most wrong. Train with Adam, then switch to L-BFGS, and see whether it helps. Add Fourier features to capture sudden changes. Use a few simulated measurements to recover an unknown number in the equation, showing how a PINN can mix physics with a little data.